

Investigating numerical solutions to the Korteweg-De Vries equation

Ojas Tiwari, November 20th 2025

Abstract

In this report, we investigate the single and double soliton solutions to the Korteweg-De Vries equation. The Korteweg-De Vries equation is used to model shallow water waves. The form we are using is in the dimensionless form as presented in Section 1.

We also investigate the motion of soliton solutions when periodic boundary conditions are used as described in the method.

1 Introduction

The key equation we are using here is the Korteweg-de-Vries (KdV) equation for waves in shallow water [1].

$$\partial_t \phi + \partial_x^3 \phi - 6\phi \partial_x \phi = 0 \quad (1)$$

In order to solve this numerically, we will be using a simple forward Euler approach. Something more stable, like Crank-Nicholson would potentially be quite difficult to apply, but we will expand on this later.

In order to apply this, we define the time step size δt , and the space step size as δx . Then the position of a point in space is represented using the index i , such that $x = i\delta x$, and for time we use the index j , so that $t = j\delta t$. Then, assuming that the step sizes are small enough relative to the system being considered, we can apply the following approximations to obtain a linear system of equations. We will also write $\phi_{i,j}$ to refer to the value of ϕ at $x = i\delta x$, and $t = j\delta t$. With some further rearrangement (see Appendix 1), we get that **TO BE COMPLETED**

Now, we see that we can quite easily compute the new value for each of the positions in the middle so long as the values to the right are known. But this means that we will need to find a condition on the right hand side that is realistic. We will try several.

1.1 Zero Value

We will first try the simplest of all, just setting all the values past the boundary to be zero. I expect that this will provide quite normal behaviour until the solitons arrive at the boundary at which point we will start to see some really quite violent behaviour due to the large gradient.

1.2 Periodic

Based on the work of Guan and Kuksin, we can see that if we set the values to loop back around, we will get a continuous system that will allow the soliton to keep moving through. For visualisation, we will display multiple copies of the system side by side. This will mean that if we can sufficiently make the box large enough, we can have a smooth playground in which to test things like multiple soliton collisions

1.3 Forced Boundary

Based on the work of Huang and Lin, we can see that some kind of forced oscillation may perhaps be worth investigating.

2 Method

For this solver, the main computational challenge is efficiently computing the matrix inverse. For this, we will be using Gaussian elimination from the `NumPy` library which is written in C++ allowing for high performance computation.

After this, we can simply iterate over the initial state to progress the simulation. We record the values in an array which is passed to `Matplotlib` for animating and plotting.

2.1 Derivative Approximations

In order to numerically approximate the values of the derivatives involved in Equation 1, we will use the first principles definition of the derivative. Specifically, for the first order derivative, it is convenient to use the "forward" derivative.

$$\partial_x \phi \approx \frac{\phi(x+h) - \phi(x)}{h} \quad (2)$$

However, for the third order derivative, we would get a "lop-sided" definition, and in our finite resolution simulation this will add a lot of numerical error. In order to reduce this, and make the derivative "more symmetric", we use an "average" derivative, and apply this thrice. The end effect, is that we sample points at $x+2h, x+h, x-h, x-2h$, which means the derivative is sensitive to differences on both sides.

$$\partial_{xxx} \approx \frac{\phi(x+2h) - 2\phi(x+h) + 2\phi(x-h) - \phi(x-2h)}{h^3} \quad (3)$$

The derivation of this result is provided in Appendix A.

2.2 Boundary Conditions

In order to simulate different boundary conditions, we create a function `phi()` which would take in an index and output a value corresponding to the height at that index.

The reason this is necessary is because the higher order derivatives will make calls to values of ϕ which are in fact outside of the array's bounds. To resolve this, the `phi()` function will still be able to return values based on some rule. The modularity of using a dedicated function to do this and passing this as a parameter to various functions means that we can very easily swap boundary conditions without having to rewrite sections of code each time.

2.2.1 Periodic Boundary Conditions

For periodic boundary conditions, we see that we can use the modulus function to cause the indices to wrap around to the start of the array.

3 Results and analysis

4 Conclusions

Appendix A

Here we will derive the discretized version of the differential equation so that we can use an explicit step method.

Firstly, we need to write the spatial derivatives. Specifically, we want to provide a second order error in the step size. To do this, we will use the central difference result.

$$\begin{aligned}\frac{\partial^3 \phi}{\partial x^3} &\approx \frac{\frac{\phi(x+2h)-2\phi(x+h)+\phi(x)}{h^2} - \frac{\phi(x)-2\phi(x-h)+\phi(x-2h)}{h^2}}{2h} \\ &= \frac{\phi(x+2h) - 2\phi(x+h) + 2\phi(x-h) - \phi(x-2h)}{2h^3}\end{aligned}$$

This has an error of order h^2 .

References

- [1] Brauer, Klaus, Lecture Notes, University of Osnabrueck <https://users.wfu.edu/natalie/f00phy711/lecturenote/kdv.pdf>